

```
In [1]: import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\pratiksha\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

Out[1]: True

```
In [2]: from nltk.tokenize import sent_tokenize, word_tokenize
```

```
text = """Natural Language Processing enables computers to understand human language.
It is used in many applications like chatbots, search engines, and translation systems.
NLTK is a popular Python library for NLP tasks."""
```

```
# Sentence Tokenization
```

```
tokenized_sentences = sent_tokenize(text)
print("Sentence Tokens:\n", tokenized_sentences)
```

```
# Word Tokenization
```

```
tokenized_words = word_tokenize(text)
print("\nWord Tokens:\n", tokenized_words)
```

Sentence Tokens:

```
['Natural Language Processing enables computers to understand human language.', 'It is used in many applications like chatbots, search engines, and translation systems.', 'NLTK is a popular Python library for NLP tasks.']
```

Word Tokens:

```
['Natural', 'Language', 'Processing', 'enables', 'computers', 'to', 'understand', 'human', 'language', '.', 'It', 'is', 'used', 'in', 'many', 'applications', 'like', 'chatbots', ',', 'search', 'engines', ',', 'and', 'translation', 'systems', '.', 'NLTK', 'is', 'a', 'popular', 'Python', 'library', 'for', 'NLP', 'tasks', '.']
```

```
In [3]: from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words("english"))
```

```
filtered_words = []
```

```
for w in tokenized_words:
    if w.lower() not in stop_words and w.isalpha(): # isalpha() removes punctuation
        filtered_words.append(w)
```

```
print("Original Words:\n", tokenized_words)
```

```
print("\nAfter Stop Word Removal:\n", filtered_words)
```

Original Words:

```
['Natural', 'Language', 'Processing', 'enables', 'computers', 'to', 'understand', 'human', 'language', '.', 'It', 'is', 'used', 'in', 'many', 'applications', 'like', 'chatbots', ',', 'search', 'engines', ',', 'and', 'translation', 'systems', '.', 'NLTK', 'is', 'a', 'popular', 'Python', 'library', 'for', 'NLP', 'tasks', '.']
```

After Stop Word Removal:

```
['Natural', 'Language', 'Processing', 'enables', 'computers', 'understand', 'human', 'language', 'used', 'many', 'applications', 'like', 'chatbots', 'search', 'engines', 'translation', 'systems', 'NLTK', 'popular', 'Python', 'library', 'NLP', 'tasks']
```

In [4]: `from nltk.stem import PorterStemmer`

```
ps = PorterStemmer()

stemmed_words = [ps.stem(w) for w in filtered_words]

print("Filtered Words:\n", filtered_words)
print("\nStemmed Words:\n", stemmed_words)
```

Filtered Words:

```
['Natural', 'Language', 'Processing', 'enables', 'computers', 'understand', 'human', 'language', 'used', 'many', 'applications', 'like', 'chatbots', 'search', 'engines', 'translation', 'systems', 'NLTK', 'popular', 'Python', 'library', 'NLP', 'tasks']
```

Stemmed Words:

```
['natur', 'languag', 'process', 'enabl', 'comput', 'understand', 'human', 'languag', 'use', 'mani', 'applic', 'like', 'chatbot', 'search', 'engin', 'translat', 'system', 'nltk', 'popular', 'python', 'librari', 'nlp', 'task']
```

In [5]: `from nltk.stem.wordnet import WordNetLemmatizer`

```
lem = WordNetLemmatizer()

lemmatized_words = [lem.lemmatize(w, "v") for w in filtered_words]

print("Filtered Words:\n", filtered_words)
print("\nLemmatized Words:\n", lemmatized_words)

# Quick comparison demo
test_words = ["flying", "running", "better", "connected", "studies"]
print("\n--- Stem vs Lemma Comparison ---")
for word in test_words:
    print(f"{word:15} | Stem: {ps.stem(word):12} | Lemma: {lem.lemmatize(word, 'v')}")
```

Filtered Words:

```
['Natural', 'Language', 'Processing', 'enables', 'computers', 'understand', 'human', 'language', 'used', 'many', 'applications', 'like', 'chatbots', 'search', 'engines', 'translation', 'systems', 'NLTK', 'popular', 'Python', 'library', 'NLP', 'tasks']
```

Lemmatized Words:

```
['Natural', 'Language', 'Processing', 'enable', 'computers', 'understand', 'human', 'language', 'use', 'many', 'applications', 'like', 'chatbots', 'search', 'engines', 'translation', 'systems', 'NLTK', 'popular', 'Python', 'library', 'NLP', 'task']
```

--- Stem vs Lemma Comparison ---

flying	Stem: fli	Lemma: fly
running	Stem: run	Lemma: run
better	Stem: better	Lemma: better
connected	Stem: connect	Lemma: connect
studies	Stem: studi	Lemma: study

In [6]: `sent = "Albert Einstein was born in Ulm Germany in 1879"`

```
tokens = nltk.word_tokenize(sent)

pos_tags = nltk.pos_tag(tokens)
print("Tokens:\n", tokens)
print("\nPOS Tags:\n", pos_tags)
```

Tag meaning reference

```
print("""
```

Common POS Tags:

```
    NN = Noun           VBD = Verb past tense
    NNP = Proper Noun   JJ = Adjective
    VBZ = Verb          RB = Adverb
    IN = Preposition    CD = Cardinal number
""")
```

Tokens:

```
['Albert', 'Einstein', 'was', 'born', 'in', 'Ulm', 'Germany', 'in', '1879']
```

POS Tags:

```
[('Albert', 'NNP'), ('Einstein', 'NNP'), ('was', 'VBD'), ('born', 'VBN'), ('in', 'IN'), ('Ulm', 'NNP'), ('Germany', 'NNP'), ('in', 'IN'), ('1879', 'CD')]
```

Common POS Tags:

NN	= Noun	VBD	= Verb past tense
NNP	= Proper Noun	JJ	= Adjective
VBZ	= Verb	RB	= Adverb
IN	= Preposition	CD	= Cardinal number

```
In [7]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn import metrics

# Sample dataset (phrase, category)
data_phrases = [
    "I love this movie it is amazing",
    "Wonderful film great acting",
    "Best movie I have ever seen",
    "Brilliant and entertaining film",
    "This movie is terrible and boring",
    "Awful film waste of time",
    "Horrible experience very bad acting",
    "Worst movie I have ever watched"
]
labels = [1, 1, 1, 1, 0, 0, 0, 0] # 1 = positive, 0 = negative

# TF-IDF Feature Extraction
tf = TfidfVectorizer()
X = tf.fit_transform(data_phrases)

print("TF-IDF Feature Shape:", X.shape)
print("Feature Names:", tf.get_feature_names_out())

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.25, random_state=42
)

# Multinomial Naive Bayes
clf = MultinomialNB().fit(X_train, y_train)
predicted = clf.predict(X_test)

print("\nPredicted Labels:", predicted)
print("Actual Labels: ", y_test)
print("Accuracy:", metrics.accuracy_score(y_test, predicted))
```

TF-IDF Feature Shape: (8, 29)

Feature Names: ['acting' 'amazing' 'and' 'awful' 'bad' 'best' 'boring' 'brilliant' 'entertaining' 'ever' 'experience' 'film' 'great' 'have' 'horrible' 'is' 'it' 'love' 'movie' 'of' 'seen' 'terrible' 'this' 'time' 'very' 'waste' 'watched' 'wonderful' 'worst']

Predicted Labels: [0 1]

Actual Labels: [1, 0]

Accuracy: 0.0

In []: